

Daniel Neugent, Mariam Oraby, Jake Bernard, Jacob Fonyi, Tanner Gurley

Objective

You have designed the logical blueprint for your database. Now, create a physical database based on this logical design and populate it with meaningful data. Although the deliverable (artifact) for this part of the project will be a Canvas submission, you must ensure you have a fully functional database. This database should be rigorously tested to verify that it meets all the requirements outlined in your project proposal.

Introduction

Project Overview: The purpose of our database is to give both library administrators and users accurate information as to what the library currently has to offer. For the end users, they want to be able to search for products such as books, magazines, and digital media. The library staff may also want to search for this media to check what they have in stock, however they additionally will get feedback based on who has checked what out in order to properly take stock of what the library should be expecting from people.

Scope: The library database will manage and track the collection of physical and digital media (Books, Magazines, DVDs, CDs, and Video Games) alongside the interactions between users and staff. End users (Library Users) will be able to search for available items to check out, while staff members (Library Staff) will have access to administrative functions such as restocking, reshelving, and tracking checkouts and returns.

The system ensures:

- Accurate cataloging of all library items with detailed metadata.
- Tracking of user borrowing history.
- Staff oversight of checkouts, returns, and restocking processes.
- Availability of search functions for both users and staff.
- Maintenance of accurate stock levels.

Glossary

Book – A physical printed work with attributes including ISBN, Title, multiple Authors, Page Count, Genre, Edition, and Quantity in stock.

Magazine: A periodical publication identified by ISBN, Title, Issue number, Publisher, and Page Count.

DVD: A digital video disc item with attributes such as Title, unique DVD_ID, Genre, Length, Actor(s), and Director.

CD: A compact disc with attributes including Title, Track listing, CD_ID, and Author.

Video Game: A digital or physical game with Name, Genre, Rating, and Release Date.

Library User: A person registered to borrow items from the library. Attributes: First Name, Last Name, Start Date, End Date, and User ID.

Library Staff: A staff member responsible for managing library resources. Attributes: First Name, Last Name, Start Date, End Date, ID, and Position.

Check Out: The process of a Library User borrowing an item (Book, DVD, CD, or Video Game).

Reshelving: The act of a Library Staff member returning an item to stock after a user has checked it out.

Restocked Date: The date when an item is returned and made available again.

Search: A function available to both Users and Staff for finding items in the library's collection.

Choose Your Platform

[5 points]: You can use the EECS Cycle servers with MariaDB, or if your team prefers, select another database product (like MySQL, PostgreSQL, or SQL Server). Be sure to explain your choice in your report. What factors influenced your decision? Familiarity with the system? Specific features? Potential limitations?

While we originally wanted to use Railway (an AWS wrapper) we decided against that because it would be free and easier to use SQLite since we all already are familiar with it, and it's easy to test on each of our machines. Just about anything we could want to do for the sake of this project could be done on our laptops here at KU, but if it were a more professional project we would use a more official service like Railway. We also chose SQLite because we had the most familiarity with it at work in our internships and out of class opportunities

Create Your Database: Create a physical database schema using your DDL scripts. Ensure all tables are correctly created with appropriate constraints (primary keys, foreign keys, etc.). Organize all SQL scripts (DDL, data population, queries, reports) clearly, name them descriptively, and include comments explaining their functions.

Print Your Physical Schema

Print the SQL DDL statements that you used to create the physical schema.

-- Enable foreign key constraint enforcement

PRAGMA foreign_keys = ON;

-- Create the Book table

```
CREATE TABLE Book (  
    ISBN TEXT PRIMARY KEY,  
    Title TEXT NOT NULL,  
    Author TEXT,  
    Page_count INTEGER,  
    genre TEXT,  
    edition TEXT,  
    Quantity INTEGER NOT NULL CHECK (Quantity >= 0)  
);
```

-- Create the Magazine table

```
CREATE TABLE Magazine (  
    ISBN TEXT PRIMARY KEY,  
    Title TEXT NOT NULL,  
    Issue INTEGER,  
    Publisher TEXT,  
    Page_count INTEGER  
);
```

-- Create the DVD table

```
CREATE TABLE DVD (  
    DVD_ID INTEGER PRIMARY KEY AUTOINCREMENT,  
    Title TEXT NOT NULL,  
    Genre TEXT,  
    Length INTEGER,  
    Actor TEXT,  
    Director TEXT  
);
```

-- Create the CD table

```
CREATE TABLE CD (  
    CD_ID INTEGER PRIMARY KEY AUTOINCREMENT,  
    Title TEXT NOT NULL,  
    Tracks INTEGER,  
    Author TEXT  
);
```

-- Create the LibraryUser table

```
CREATE TABLE LibraryUser (  
    User_ID INTEGER PRIMARY KEY AUTOINCREMENT,  
    FirstName TEXT NOT NULL,  
    LastName TEXT NOT NULL,  
    Start_date DATE,  
    end_date DATE,  
    ID TEXT UNIQUE  
);
```

-- Create the LibraryStaff table

```
CREATE TABLE LibraryStaff (  
    Staff_ID INTEGER PRIMARY KEY AUTOINCREMENT,  
    FirstName TEXT NOT NULL,  
    LastName TEXT NOT NULL,  
    Start_date DATE,  
    end_date DATE,  
    ID TEXT UNIQUE,  
    Position TEXT  
);
```

-- Create the VideoGames table

```
CREATE TABLE VideoGames (  
    Name TEXT PRIMARY KEY,  
    genre TEXT,  
    rating TEXT,  
    release_date DATE  
);
```

-- Create the Book_Checkout table to manage book checkouts

-- The composite primary key ensures a user can check out a specific book up to 5 times.

```
CREATE TABLE Book_Checkout (  
    User_UID INTEGER,  
    Book_ISBN TEXT,
```

```
Checkout_Number INTEGER CHECK (Checkout_Number > 0 AND
Checkout_Number <= 5),
Checkout_Date DATE,
Due_Date DATE,
PRIMARY KEY (User_UID, Book_ISBN, Checkout_Number),
FOREIGN KEY (User_UID) REFERENCES LibraryUser(User_ID),
FOREIGN KEY (Book_ISBN) REFERENCES Book(ISBN)
);
```

-- Create the CD_Checkout table to manage CD checkouts

```
CREATE TABLE CD_Checkout (
User_UID INTEGER,
CD_ID INTEGER,
Checkout_Number INTEGER CHECK (Checkout_Number > 0 AND
Checkout_Number <= 5),
Checkout_Date DATE,
Due_Date DATE,
PRIMARY KEY (User_UID, CD_ID, Checkout_Number),
FOREIGN KEY (User_UID) REFERENCES LibraryUser(User_ID),
FOREIGN KEY (CD_ID) REFERENCES CD(CD_ID)
);
```

-- Create the DVD_Checkout table to manage DVD checkouts

```
CREATE TABLE DVD_Checkout (
User_UID INTEGER,
DVD_ID INTEGER,
```

```
Checkout_Number INTEGER CHECK (Checkout_Number > 0 AND
Checkout_Number <= 5),
Checkout_Date DATE,
Due_Date DATE,
PRIMARY KEY (User_UID, DVD_ID, Checkout_Number),
FOREIGN KEY (User_UID) REFERENCES LibraryUser(User_ID),
FOREIGN KEY (DVD_ID) REFERENCES DVD(DVD_ID)
);
```

-- Create the VideoGame_Checkout table to manage video game checkouts

```
CREATE TABLE VideoGame_Checkout (
User_UID INTEGER,
Game_Name TEXT,
Checkout_Number INTEGER CHECK (Checkout_Number > 0 AND
Checkout_Number <= 5),
Checkout_Date DATE,
Due_Date DATE,
PRIMARY KEY (User_UID, Game_Name, Checkout_Number),
FOREIGN KEY (User_UID) REFERENCES LibraryUser(User_ID),
FOREIGN KEY (Game_Name) REFERENCES VideoGames(Name)
);
```

-- Create the Magazine_Checkout table to manage magazine checkouts

-- The composite primary key ensures a user can check out a specific magazine up to 5 times.

```
CREATE TABLE Magazine_Checkout (
User_UID INTEGER,
Magazine_ISBN TEXT,
```

```
Checkout_Number INTEGER CHECK (Checkout_Number > 0 AND
Checkout_Number <= 5),
Checkout_Date DATE,
Due_Date DATE,
PRIMARY KEY (User_UID, Magazine_ISBN, Checkout_Number),
FOREIGN KEY (User_UID) REFERENCES LibraryUser(User_ID),
FOREIGN KEY (Magazine_ISBN) REFERENCES Magazine(ISBN)
);
```

```
-- Create the Staff_Reshelves table to track items reshelfed by staff
-- One staff member can restock many items.
```

```
CREATE TABLE Staff_Reshelves (
Reshelve_ID INTEGER PRIMARY KEY AUTOINCREMENT,
Staff_UID INTEGER,
Book_ISBN TEXT,
DVD_ID INTEGER,
CD_ID INTEGER,
Game_Name TEXT,
Magazine_ISBN TEXT,
Reshelve_Date DATE,
FOREIGN KEY (Staff_UID) REFERENCES LibraryStaff(Staff_ID),
FOREIGN KEY (Book_ISBN) REFERENCES Book(ISBN),
FOREIGN KEY (DVD_ID) REFERENCES DVD(DVD_ID),
FOREIGN KEY (CD_ID) REFERENCES CD(CD_ID),
FOREIGN KEY (Game_Name) REFERENCES VideoGames(Name),
FOREIGN KEY (Magazine_ISBN) REFERENCES Magazine(ISBN)
```


);

Data Population.

Use tools or scripts to create realistic data matching the type of information your database will hold. Utilize public datasets where available, or manually enter data if your project is small. Use bulk data loading utilities for larger datasets. Validate your data for accuracy and consistency with your design.

Printing Table Contents [10 points]. Once the tables are populated, print the contents of each table to provide a quick overview of the table contents, allowing examination of data size and composition. You may use the following SQL command for each table:

```
SELECT * FROM TableName;
```

See additional commands below.

```
.mode column
```

```
.headers on
```

```
SELECT '--- Book Table ---';
```

```
SELECT * FROM Book;
```

```
SELECT '--- Magazine Table ---';
```

```
SELECT * FROM Magazine;
```

```
SELECT '--- DVD Table ---';
```

```
SELECT * FROM DVD;
```

```
SELECT '--- CD Table ---';
```

```
SELECT * FROM CD;
```

```
SELECT '--- LibraryUser Table ---';
```

```
SELECT * FROM LibraryUser;
```

```
SELECT '--- LibraryStaff Table ---';
```

```
SELECT * FROM LibraryStaff;
```

```
SELECT '--- VideoGames Table ---';
```

```
SELECT * FROM VideoGames;
```

```
SELECT '--- Book_Checkout Table ---';
```

```
SELECT * FROM Book_Checkout;
```

```
SELECT '--- CD_Checkout Table ---';
```

```
SELECT * FROM CD_Checkout;
```

```
SELECT '--- DVD_Checkout Table ---';
```

```
SELECT * FROM DVD_Checkout;
```

```
SELECT '--- VideoGame_Checkout Table ---';
```

```
SELECT * FROM VideoGame_Checkout;
```

```
SELECT '--- Magazine_Checkout Table ---';
```

```
SELECT * FROM Magazine_Checkout;
```

```
SELECT '--- Staff_Reshelves Table ---';
```

```
SELECT * FROM Staff_Reshelves;
```

I will put a few screenshots here from my terminal as examples of it running, however printing the entire database would obviously extend this doc to be far longer than what is readable.

--- DVD Table ---					
DVD_ID	Title	Genre	Length	Actor	Director
1305510779	The Lego Movie 2: The Second Part	Animation	107	Chris Pratt	Mike Mitchell
1890474302	The Emoji Movie	Animation	86	T.J. Miller	Tony Leondis
1981986063	The Simpsons Movie	Animation	87	Dan Castellana	David Silverman
2000432192	A Goofy Movie	Animation	78	Bill Farmer	Kevin Lima
2411357837	Superhero Movie	Action	75	Drake Bell	Craig Mazin
2457498945	The Super Mario Bros. Movie	Animation	92	Chris Pratt	Aaron Horvath, Michael Jelenic, Pierre Leduc
2658128330	Epic Movie	Adventure	86	Kal Penn	Jason Friedberg, Aaron Seltzer
2786362924	Date Movie	Comedy	83	Alyson Hannigan	Aaron Seltzer, Jason Friedberg
3171962020	A Minecraft Movie	Action	101	Jason Momoa	Jared Hess
3878147559	A Silent Voice: The Movie	Animation	130	Miyu Irino	Naoko Yamada
4045604496	The SpongeBob Movie: Sponge Out of Water	Animation	92	Tom Kenny	Paul Tibbitt, Mike Mitchell
4256821752	Scary Movie 3	Comedy	84	Anna Faris	David Zucker
4440282195	The Lego Batman Movie	Animation	104	Will Arnett	Chris McKay
4653366573	Not Another Teen Movie	Comedy	89	Chyler Leigh	Joel Gallen
4854974940	Garfield: The Movie	Adventure	80	Breckin Meyer	Peter Hewitt
5284383357	Jackass: The Movie	Documentary	85	Johnny Knoxville	Jeff Tremaine
5373139266	Demon Slayer: Kimetsu no Yaiba - The Movie: Mugen Train	Animation	117	Natsuki Hanae	Haruo Sotozaki
5740548985	Scary Movie	Comedy	88	Anna Faris	Keenen Ivory Wayans
6450111105	Scary Movie V	Comedy	86	Simon Rex	Malcolm D. Lee, David Zucker
6929922302	The Angry Birds Movie	Animation	97	Jason Sudeikis	Clay Kaytis, Fergal Reilly
7080716978	Cowboy Bebop: The Movie	Animation	115	Beau Billingslea	Shin'ichirō Watanabe, Tensai Okamura, Hiroyuki Okiura
7128766141	The Peanuts Movie	Animation	88	Noah Schnapp	Steve Martino
7518098849	Scary Movie 2	Comedy	83	Anna Faris	Keenen Ivory Wayans
7570570763	The SpongeBob SquarePants Movie	Animation	87	Tom Kenny	Stephen Hillenburg, Mark Osborne

--- DVD_Checkout Table ---				
User_UID	DVD_ID	Checkout_Number	Checkout_Date	Due_Date
U2022-JFKREW	3878147559	1	2023-04-21	2023-05-10
U2022-JFKREW	7518098849	1	2023-12-22	2024-01-16
U2022-JFKREW	7518098849	2	2024-05-16	2024-06-11
U2023-NAUJIT	8352542379	1	2024-02-10	2024-03-06
U2023-NAUJIT	6929922302	1	2023-07-11	2023-08-01
U2021-AGMRRE	4256821752	1	2024-07-08	2024-07-29
U2021-AGMRRE	4256821752	2	2023-11-07	2023-12-05
U2021-AGMRRE	2411357837	1	2023-04-08	2023-04-28
U2021-AGMRRE	2411357837	2	2023-01-09	2023-01-23
U2021-AGMRRE	2411357837	3	2024-05-24	2024-06-11
U2022-CRTJDK	2457498945	1	2023-01-28	2023-02-18
U2022-CRTJDK	7518098849	1	2023-03-06	2023-03-13
U2022-CRTJDK	7518098849	2	2024-02-11	2024-03-03
U2022-CRTJDK	7518098849	3	2023-10-28	2023-11-10
U2021-OXVMAY	4854974940	1	2023-03-30	2023-04-15
U2021-OXVMAY	4854974940	2	2024-04-22	2024-05-07
U2021-OXVMAY	4854974940	3	2024-03-02	2024-03-10
U2021-OXVMAY	3878147559	1	2023-10-05	2023-10-21
U2021-OXVMAY	3878147559	2	2023-12-28	2024-01-09
U2022-ZBZSWV	7128766141	1	2024-07-15	2024-07-29
U2022-ZBZSWV	7128766141	2	2024-01-12	2024-01-23
U2022-ZBZSWV	7518098849	1	2023-07-15	2023-08-14
U2022-ZBZSWV	7518098849	2	2023-05-23	2023-06-10
U2022-ZBZSWV	7518098849	3	2024-05-15	2024-06-07
U2024-XVYLPI	6929922302	1	2024-04-17	2024-04-29
U2024-XVYLPI	6929922302	2	2023-04-01	2023-04-19
U2024-XVYLPI	6929922302	3	2023-01-05	2023-02-03
U2025-ZWLOMK	4653366573	1	2023-04-27	2023-05-27

--- CD Table ---				
CD_ID	Title	Tracks	Author	
1233788834	Totally Unplugged	16	Larry Norman	
1312033141	Bop Till You Drop	15	Ry Cooder	
1398436740	Third Day	19	Third Day	
1729210362	Tracks	10	Bruce Springsteen	
1748779870	Travel-Log	11	J.J. Cale	
2067754386	.hack//SIGN Original Soundtrack 2	7	Yuki Kajiura	
2238644651	Sunwheel Dance	15	Bruce Cockburn	
2559339861	Beyond the Valley of the Proles	15	Snog	
2716579317	Pure Gold	13	Eddy Arnold	
2721226138	The Slide Area	19	Ry Cooder	
2891391463	Super Secret	13	Ookla the Mok	
2909143955	Tattoo You	10	The Rolling Stones	
2983856586	Boomer's Story	12	Ry Cooder	
3111931441	Star Trek: The Next Generation: Original Soundtrack Recordings	20	Jay Chattaway	
3265282127	Tell All Your Friends	19	Taking Back Sunday	
3605307069	Grace	13	Tribes of Neurot	
3640985664	Red Dirt Road	13	Brooks & Dunn	
3776945568	Peter Gabriel	9	Peter Gabriel	
4494755621	Plays Live	20	Peter Gabriel	
4566253510	Stunt	7	Barenaked Ladies	
4847791715	Save You	13	Pearl Jam	
4929796020	Days for Days	15	The Loud Family	
4948731002	The Battle for Everything	14	Five for Fighting	
5349026140	Dear Catastrophe Waitress	12	Belle & Sebastian	

--- LibraryUser Table ---					
User_ID	FirstName	LastName	Start_date	end_date	ID
1	Tiffany	Clark	2023-01-12	2023-06-04	U2023-QWKQID
2	Alice	Russell	2020-11-05	2024-06-06	U2020-QTSMNX
3	Margaret	White	2025-02-24	2027-09-24	U2025-WYAXPJ
4	John	Vasquez	2022-12-04	2026-07-29	U2022-UDELDK
5	Jose	Burgess	2023-06-16	2026-08-17	U2023-XNUTWE
6	Maria	Thompson	2025-04-10	2026-01-13	U2025-IIANXW
7	Brittney	Robertson	2023-09-04	2027-04-02	U2023-YOPLAW
8	Daniel	Vaughn	2024-08-18	2027-06-29	U2024-VISFAQ
9	Alicia	Simmons	2021-12-28	2026-01-13	U2021-LCVQSV
10	Audrey	Martinez	2021-10-18	2022-12-23	U2021-SQYJSU
11	Edward	Hernandez	2023-06-05	2025-08-19	U2023-SNFJOF
12	Kelli	Walker	2025-08-10	2029-04-08	U2025-XTTBQF
13	Michelle	Nash	2022-07-02	2023-12-20	U2022-QVCYMH
14	Brandi	Wilson	2021-04-02	2022-06-02	U2021-OVYGAH
15	Nicole	Santiago	2025-02-10	2027-09-18	U2025-URHWSF
16	Hannah	Knight	2023-11-21	2024-02-03	U2023-LJ00WR
17	Heidi	Erickson	2022-05-19	2026-05-06	U2022-ZBZSMV
18	Clinton	Davidson	2023-06-12	2025-04-16	U2023-UMCCJS
19	Tracy	Hernandez	2023-05-07	2026-01-30	U2023-JCITCW
20	Bryan	Wilkins	2023-05-13	2024-02-07	U2023-QHSMXP
21	Donald	Owens	2022-12-09	2024-10-21	U2022-VXYGHH
22	Stephen	Waters	2024-12-30	2029-02-04	U2024-WPAROU
23	Angela	Bass	2021-08-05	2022-03-12	U2021-EGGNUS
24	John	Wilcox	2021-08-18	2024-01-28	U2021-AALNOE
25	Paul	Smith	2023-06-16	2023-01-08	U2023-BLWQYV

--- LibraryStaff Table ---						
Staff_ID	FirstName	LastName	Start_date	end_date	ID	Position
1	Ryan	Cox	2023-12-24	2031-09-26	S2023-SHKPMQ	Librarian
2	John	Sanchez	2016-08-23	2021-10-23	S2016-QUBPME	Reference Assistant
3	David	Luna	2018-01-25	2019-12-13	S2018-VBKXAI	Technician
4	Cathy	Briggs	2019-03-28	2020-01-21	S2019-CCALNA	Library Clerk
5	Gina	Fisher	2022-08-24	2032-05-31	S2022-OCWVDJ	Library Clerk
6	Mary	Morris	2024-10-22	2032-09-12	S2024-BEWDJSV	Reference Assistant
7	Patricia	Hebert	2017-10-27	2027-09-19	S2017-EFOHDA	Reference Assistant
8	Monica	Brown	2022-03-02	2028-02-10	S2022-XFYKBH	Librarian
9	Karen	Robertson	2017-04-26	2022-10-18	S2017-IJUUCHD	Technician
10	Lisa	Mayer	2019-03-02	2024-02-04	S2019-LPGXQJ	Archivist
11	Jeffery	Horton	2025-01-02	2029-03-31	S2025-QSDHCI	Library Clerk
12	Chris	Larson	2022-04-05	2029-06-04	S2022-WFODUC	Librarian
13	Brenda	Cowan	2016-03-27	2023-07-02	S2016-WQGTKA	Archivist
14	Stephen	Campbell	2024-08-30	2034-08-19	S2024-SGQNYK	Cataloger
15	Sandra	Wise	2017-07-09	2019-07-02	S2017-BKTIAGL	Librarian

--- VideoGames Table ---				
Name	Genre	Rating	Release_date	
Grand Theft Auto V	Action	A0	2013-09-17	
The Witcher 3: Wild Hunt	Action	M	2015-05-18	
Portal 2	Shooter	M	2011-04-18	
Counter-Strike: Global Offensive	Shooter	T	2012-08-21	
Tomb Raider (2013)	Action	A0	2013-03-05	
Portal	Action	M	2007-10-09	
Left 4 Dead 2	Action	E	2009-11-17	
The Elder Scrolls V: Skyrim	Action	A0	2011-11-11	
Red Dead Redemption 2	Action	E10+	2018-10-26	
BioShock Infinite	Action	M	2013-03-26	
Half-Life 2	Action	T	2004-11-16	
Borderlands 2	Action	T	2012-09-18	
Life is Strange	Adventure	A0	2015-01-29	
BioShock	Action	E	2007-08-21	
Destiny 2	Action	M	2017-09-06	
God of War (2018)	Action	E10+	2018-04-20	
Fallout 4	Action	E10+	2015-11-09	
PAYDAY 2	Action	E	2013-08-13	
Limbo	Action	E	2010-07-21	
Team Fortress 2	Action	E	2007-10-10	

Functionality Testing.

Develop SQL queries to store and update data in your database, implement functionalities described in your requirements document (e.g., searching for specific records, generating reports), and create clear, useful reports. Test to ensure:

- Queries and reports produce correct results.
- Database handles unexpected input or edge cases without crashing.
- Database performs well (queries are not excessively slow).

For the sake of making things easier to read, all functionality testing can be found at the following links in our GitHub:

1. https://github.com/l33tdaniel/databases-447-queryreaders/blob/main/test_checkout_limits.sql
2. https://github.com/l33tdaniel/databases-447-queryreaders/blob/main/import_data.sh
3. https://github.com/l33tdaniel/databases-447-queryreaders/blob/main/functionality_tests.sql

If you look through this file, you're able to see the entirety of the code that we use in order to populate our database with the seed data. Additionally, all data can be found in the "csvs" folder in our GitHub as well. We test functionality such as whether or not you can checkout more than what you should be able to along with inserting into and reading essential items from the database.